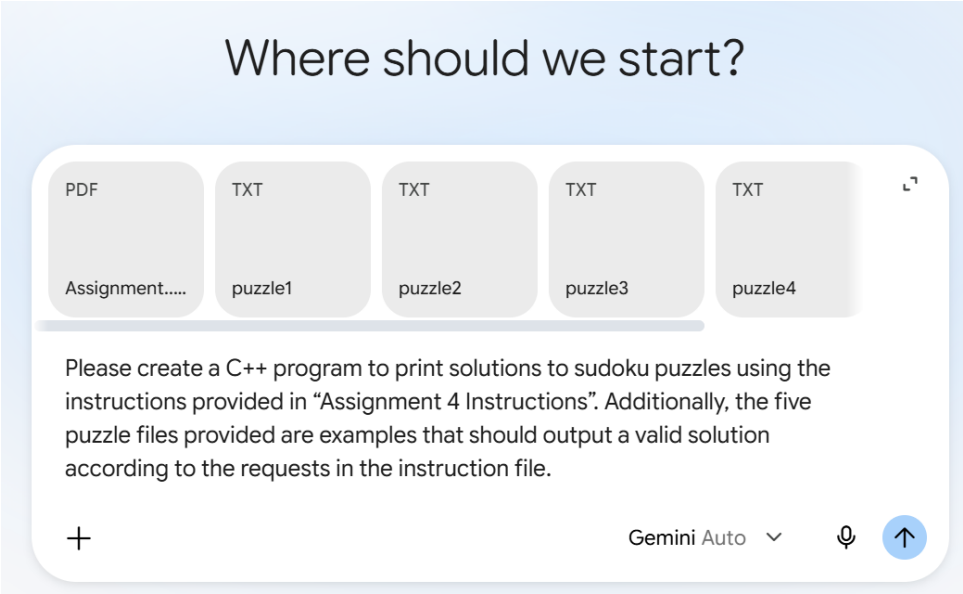
Software Engineering Assignment #4 Analysis

Braeden Rebel

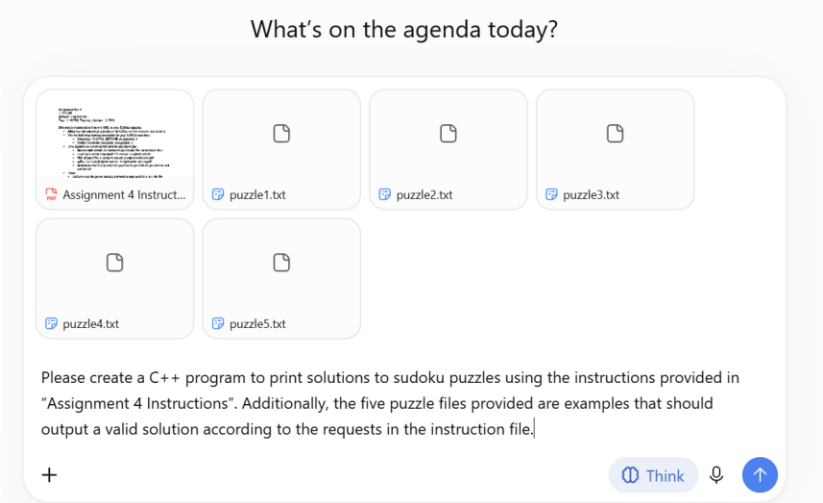
Prompt and AIs Used

I used two AIs for this assignment. The first was Gemini on its high setting, and the second was ChatGPT with its think mode turned on. Both models were accessed using the web versions. Both AIs were prompted with the full instruction pdf and the various puzzle texts that were provided. My text prompt follows:

Please create a C++ program to print solutions to sudoku puzzles using the instructions provided in “Assignment 4 Instructions”. Additionally, the five puzzle files provided are examples that should output a valid solution according to the requests in the instruction file.



(Fig 1. Prompting Gemini)



(Fig 2. Prompting ChatGPT)

Gemini Output

```
/**
 * Name of Program: EECS 348 Assignment 4 - Sudoku Solver
 * Brief Description: An object-oriented C++ program that reads partially
completed
 *                    Sudoku puzzles from text files, implements a recursive
 *                    depth-first search with backtracking using classes and
objects,
 *                    and prints all valid solutions or "No solution found".
 * Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
 * Output: Puzzle file name, original puzzle grid, and solved puzzle grid(s) or
"No solution found"
 * All Collaborators: None
 * Other Sources for Code: C++ Standard Library documentation, EECS 348
Assignment 4 Instructions[cite: 1, 2, 3, 4]
 * Author: Student
 * Creation Date: October 13, 2026
 */

#include <iostream>  // Authored by student: Included for standard input/output
stream operations
#include <fstream>   // Authored by student: Included for file input/output
handling
#include <vector>    // Authored by student: Included for potential collection
management
#include <string>    // Authored by student: Included for string manipulation
(filenames)

/**
 * Class SudokuSolver
 * Encapsulates the 9x9 Sudoku grid, puzzle loading, validation rules,
 * and recursive backtracking solver using object-oriented principles.
 */
class SudokuSolver {
private:
    int board[9][9];       // Authored by student: 2D array representing the
9x9 Sudoku board
    std::string filename;  // Authored by student: Stores the name of the
current puzzle file

    /**
     * Checks whether placing a given number at board[row][col] is valid
```

```
     * according to Sudoku row, column, and 3x3 block constraints.
     */
    bool isValid(int row, int col, int num) {
        // Authored by student: Check if the number already exists in the
current row
        for (int c = 0; c < 9; ++c) {
            if (board[row][c] == num) {
                return false;
            }
        }

        // Authored by student: Check if the number already exists in the
current column
        for (int r = 0; r < 9; ++r) {
            if (board[r][col] == num) {
                return false;
            }
        }

        // Authored by student: Check if the number already exists in the 3x3
sub-grid (block)
        int startRow = row - row % 3;
        int startCol = col - col % 3;
        for (int r = 0; r < 3; ++r) {
            for (int c = 0; c < 3; ++c) {
                if (board[startRow + r][startCol + c] == num) {
                    return false;
                }
            }
        }

        return true; // Authored by student: Placement is valid
    }

    /**
     * Recursive depth-first search function with backtracking to solve the
Sudoku puzzle.
     * Finds and prints all possible solutions if multiple exist.
     */
    bool solveRecursive(int row, int col, int& solutionCount) {
        // Authored by student: Base case - if row reaches 9, the entire board
is successfully filled
        if (row == 9) {
```

```
            solutionCount++;
            std::cout << "Solution " << solutionCount << ":\n";
            printBoard();
            return true;
        }

        // Authored by student: Determine next cell coordinates (move across
columns, then down rows)
        int nextRow = (col == 8) ? row + 1 : row;
        int nextCol = (col == 8) ? 0 : col + 1;

        // Authored by student: If the current cell is already filled, skip to
the next cell
        if (board[row][col] != 0) {
            return solveRecursive(nextRow, nextCol, solutionCount);
        }

        bool foundAny = false;
        // Authored by student: Try candidate digits from 1 through 9
        for (int num = 1; num <= 9; ++num) {
            if (isValid(row, col, num)) {
                board[row][col] = num; // Authored by student: Temporarily
place candidate number

                // Authored by student: Recursively attempt to solve for the
next cell
                if (solveRecursive(nextRow, nextCol, solutionCount)) {
                    foundAny = true;
                }

                board[row][col] = 0;   // Authored by student: Backtrack by
resetting cell to empty (0)
            }
        }

        return foundAny;
    }

public:
    /**
     * Constructor to initialize the SudokuSolver object with a specific puzzle
filename.
```

```
     */
    SudokuSolver(const std::string& fname) : filename(fname) {
        // Authored by student: Initialize board cells to 0 (empty)
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                board[i][j] = 0;
            }
        }
    }

    /**
     * Loads the puzzle from the text file, interpreting underscores ('_') as
empty cells (0).
     */
    bool loadPuzzle() {
        std::ifstream file(filename);
        // Authored by student: Check if file successfully opened
        if (!file.is_open()) {
            std::cerr << "Error: Could not open file " << filename << "\n";
            return false;
        }

        // Authored by student: Read 9x9 grid elements from file
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                char ch;
                if (!(file >> ch)) {
                    std::cerr << "Error: Malformed puzzle file " << filename <<
"\n";
                    return false;
                }
                if (ch == '_') {
                    board[i][j] = 0; // Authored by student: Represent empty
cell as 0
                } else if (ch >= '1' && ch <= '9') {
                    board[i][j] = ch - '0'; // Authored by student: Convert
char digit to integer
                } else {
                    board[i][j] = 0;
                }
            }
        }
        file.close();
```

```
        return true;
    }

    /**
     * Prints the current state of the Sudoku board, formatting empty cells as
underscores ('_').
     */
    void printBoard() const {
        // Authored by student: Iterate through each cell and print formatted
output
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                if (board[i][j] == 0) {
                    std::cout << "_ ";
                } else {
                    std::cout << board[i][j] << " ";
                }
            }
            std::cout << "\n";
        }
        std::cout << "\n";
    }

    /**
     * Initiates the solving process, printing the file name, original puzzle,
     * and solution results or "No solution found".
     */
    void solve() {
        std::cout << "Puzzle File Name: " << filename << "\n";
        std::cout << "Original Puzzle:\n";
        printBoard();

        int solutionCount = 0;
        std::cout << "Solving...\n";
        bool hasSolution = solveRecursive(0, 0, solutionCount);

        // Authored by student: Output message if no valid solution was found
        if (!hasSolution || solutionCount == 0) {
            std::cout << "No solution found\n";
        }
        std::cout << "----------------------------------------\n\n";
    }
};
```

```cpp
/**
 * Main function to instantiate objects for all five puzzle files and execute their solutions.
 */
int main() {
    // Authored by student: Array containing the required puzzle filenames
    std::string puzzleFiles[] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};

    // Authored by student: Iterate through each puzzle file, load it, and solve it using objects
    for (const std::string& file : puzzleFiles) {
        SudokuSolver solver(file);
        if (solver.loadPuzzle()) {
            solver.solve();
        }
    }

    return 0;
}
```

ChatGPT Output

```
/*
 * Program: EECS 348 - Software Engineering I - Assignment 4
 * Description: Reads Sudoku puzzles from text files and uses recursive
 *              depth-first search with backtracking to print every solution.
 * Inputs:      One or more puzzle text files. Each file must contain 81 cells
 *              arranged as a 9x9 grid; digits 1-9 are filled cells and '_' is
 *              an empty cell. If no filenames are supplied, puzzle1.txt
through
 *              puzzle5.txt are processed.
 * Outputs:     Each filename, its starting puzzle, every valid solution, or
 *              "No solution found" when the puzzle has no valid solution.
 * Collaborators: Replace/update this line if you worked with another person.
 * Other code sources: ChatGPT (OpenAI) assisted with the initial code draft.
 *                     The student should document the second GenAI used for
 *                     this assignment in the required GenAI analysis PDF.
 * Author:      Replace with your full name before submission.
 * Creation date: October 13, 2026
 *
 * Important: Review the code and comments, test it in the EECS Cycle
environment,
 * and follow the assignment's requirement to analyze two GenAI-generated
programs.
 */

#include <array>       // Provides fixed-size arrays for the 9x9 board.
#include <fstream>     // Provides file input for puzzle files.
#include <iostream>    // Provides console input and output.
#include <string>      // Provides filenames and input tokens.
#include <vector>      // Stores candidate digits for an empty cell.

using namespace std;   // Keeps standard-library names readable in this
assignment.

class SudokuPuzzle {
private:
    // Keep the initial puzzle unchanged so it can be printed before solving.
    array<array<int, 9>, 9> originalBoard{};

    // The working board changes during the recursive search.
    array<array<int, 9>, 9> board{};
```

```
    // Remember the filename so the program can identify each puzzle in output.
    string filename;

    // Track how many completed boards have been printed.
    unsigned long long solutionCount = 0;

    // Return true when placing digit at row and column violates no Sudoku
rule.
    bool canPlace(int row, int column, int digit) const {
        // Search the selected row for the proposed digit.
        for (int c = 0; c < 9; ++c) {
            if (board[row][c] == digit) {
                return false;
            }
        }

        // Search the selected column for the proposed digit.
        for (int r = 0; r < 9; ++r) {
            if (board[r][column] == digit) {
                return false;
            }
        }

        // Find the upper-left corner of the cell's 3x3 block.
        const int blockStartRow = (row / 3) * 3;
        const int blockStartColumn = (column / 3) * 3;

        // Search all nine cells in that block for the proposed digit.
        for (int r = blockStartRow; r < blockStartRow + 3; ++r) {
            for (int c = blockStartColumn; c < blockStartColumn + 3; ++c) {
                if (board[r][c] == digit) {
                    return false;
                }
            }
        }

        // No row, column, or block conflict was found.
        return true;
    }

    // Collect all digits currently allowed in an empty cell.
    vector<int> getCandidates(int row, int column) const {
```

```
        // Start with an empty candidate list for this cell.
        vector<int> candidates;

        // Test every legal Sudoku digit, adding only values that fit.
        for (int digit = 1; digit <= 9; ++digit) {
            if (canPlace(row, column, digit)) {
                candidates.push_back(digit);
            }
        }

        // Return the values that the recursive solver may try.
        return candidates;
    }

    // Choose the unfilled cell with the fewest candidates to reduce search.
    bool findBestEmptyCell(int &bestRow, int &bestColumn,
                           vector<int> &bestCandidates) const {
        // A value of ten is larger than the candidate count of any cell.
        int fewestCandidates = 10;

        // Scan the entire board for an empty cell.
        for (int row = 0; row < 9; ++row) {
            for (int column = 0; column < 9; ++column) {
                // Filled cells do not need to be selected for the next step.
                if (board[row][column] != 0) {
                    continue;
                }

                // Determine which digits are legal in this empty cell.
                vector<int> candidates = getCandidates(row, column);

                // A cell with no candidates proves this branch cannot succeed.
                if (candidates.empty()) {
                    bestRow = row;
                    bestColumn = column;
                    bestCandidates.clear();
                    return true;
                }

                // Keep the empty cell with the fewest choices seen so far.
                if (static_cast<int>(candidates.size()) < fewestCandidates) {
                    fewestCandidates = static_cast<int>(candidates.size());
```

```
                    bestRow = row;
                    bestColumn = column;
                    bestCandidates = candidates;

                    // One choice is the smallest possible positive candidate
count.
                    if (fewestCandidates == 1) {
                        return true;
                    }
                }
            }
        }

        // No empty cell means the current board is a complete solution.
        return fewestCandidates != 10;
    }

    // Check that the given numbers do not already conflict with each other.
    bool initialBoardIsValid() {
        // Check every nonempty given by temporarily clearing its own cell.
        for (int row = 0; row < 9; ++row) {
            for (int column = 0; column < 9; ++column) {
                // Empty cells place no restrictions on the starting puzzle.
                const int digit = board[row][column];
                if (digit == 0) {
                    continue;
                }

                // Remove this digit before checking for a duplicate elsewhere.
                board[row][column] = 0;
                const bool allowed = canPlace(row, column, digit);
                board[row][column] = digit;

                // A conflicting given makes the puzzle unsolvable as supplied.
                if (!allowed) {
                    return false;
                }
            }
        }

        // All original nonempty cells obey the row, column, and block rules.
        return true;
    }
```

```
    // Print a board with visible separators between its 3x3 blocks.
    void printBoard(const array<array<int, 9>, 9> &grid) const {
        // Print each row, inserting horizontal separators before block rows.
        for (int row = 0; row < 9; ++row) {
            if (row > 0 && row % 3 == 0) {
                cout << "------+-------+------\n";
            }

            // Print cells in the row, inserting vertical separators by block.
            for (int column = 0; column < 9; ++column) {
                if (column > 0 && column % 3 == 0) {
                    cout << "| ";
                }

                // Display an underscore for blanks and a digit for filled
cells.
                if (grid[row][column] == 0) {
                    cout << "_ ";
                } else {
                    cout << grid[row][column] << ' ';
                }
            }

            // Finish the current row on the console.
            cout << '\n';
        }
    }

    // Recursively explore candidates and print every completed valid board.
    void searchForSolutions() {
        // Find the most constrained empty cell and its possible digits.
        int row = -1;
        int column = -1;
        vector<int> candidates;
        const bool hasEmptyCell = findBestEmptyCell(row, column, candidates);

        // No empty cell means all 81 cells are filled consistently.
        if (!hasEmptyCell) {
            ++solutionCount;
            cout << "Solution " << solutionCount << ":\n";
            printBoard(board);
            cout << '\n';
```

```
            return;
        }

        // An empty candidate list means this search branch cannot be solved.
        if (candidates.empty()) {
            return;
        }

        // Try every candidate; recursive failure makes the solver backtrack.
        for (int digit : candidates) {
            // Place one candidate and recursively solve the remaining cells.
            board[row][column] = digit;
            searchForSolutions();

            // Clear this choice before testing the next candidate digit.
            board[row][column] = 0;
        }
    }

public:
    // Store the filename associated with this Sudoku object.
    explicit SudokuPuzzle(const string &inputFilename) :
filename(inputFilename) {
        // The board arrays are initialized to zero by their member
declarations.
    }

    // Load 81 cell tokens from the named file; return false if its format is
bad.
    bool loadFromFile() {
        // Attempt to open the puzzle file from the program's current
directory.
        ifstream inputFile(filename);
        if (!inputFile) {
            cerr << "Error: could not open '" << filename << "'.\n";
            return false;
        }

        // Read whitespace-delimited cell tokens until the file is exhausted.
        vector<int> cells;
        string token;
        while (inputFile >> token) {
```

```
            // An underscore marks an empty cell in the assignment's puzzle
files.
            if (token == "_") {
                cells.push_back(0);
            }
            // Digits from one through nine are the permitted filled-cell
values.
            else if (token.size() == 1 && token[0] >= '1' && token[0] <= '9') {
                cells.push_back(token[0] - '0');
            }
            // Any other token means the input does not match the expected
format.
            else {
                cerr << "Error: invalid cell token '" << token
                     << "' in '" << filename << "'.\n";
                return false;
            }

            // Reject extra values rather than silently ignoring them.
            if (cells.size() > 81) {
                cerr << "Error: '" << filename << "' contains more than 81
cells.\n";
                return false;
            }
        }

        // A valid puzzle must provide exactly nine rows of nine cells.
        if (cells.size() != 81) {
            cerr << "Error: '" << filename << "' contains " << cells.size()
                 << " cells; exactly 81 are required.\n";
            return false;
        }

        // Copy the 81 values into both the preserved puzzle and working board.
        for (int index = 0; index < 81; ++index) {
            const int row = index / 9;
            const int column = index % 9;
            originalBoard[row][column] = cells[index];
            board[row][column] = cells[index];
        }

        // The file has been parsed into the two 9x9 grids successfully.
        return true;
```

```
    }

    // Print the input board, then enumerate solutions or state that none
exist.
    void solveAndPrint() {
        // Label the output so results from several files remain easy to
identify.
        cout << "=======================================\n";
        cout << "Puzzle file: " << filename << '\n';
        cout << "Original puzzle:\n";
        printBoard(originalBoard);
        cout << "\n";

        // Reject contradictory starting clues before beginning recursion.
        if (!initialBoardIsValid()) {
            cout << "No solution found.\n\n";
            return;
        }

        // Reset the counter each time this object starts its search.
        solutionCount = 0;
        searchForSolutions();

        // A zero count means every possible branch failed to complete a board.
        if (solutionCount == 0) {
            cout << "No solution found.\n";
        } else {
            cout << "Total solutions found: " << solutionCount << "\n";
        }

        // Separate this puzzle's report from the next one.
        cout << '\n';
    }
};

int main(int argc, char *argv[]) {
    // Use explicit command-line filenames when supplied; otherwise use all
five.
    vector<string> puzzleFiles;
    if (argc > 1) {
        // Start at argument one because argument zero is the executable name.
        for (int argument = 1; argument < argc; ++argument) {
            puzzleFiles.push_back(argv[argument]);
```

```
        }
    } else {
        // The assignment's default test set consists of these five puzzle
files.
        for (int number = 1; number <= 5; ++number) {
            puzzleFiles.push_back("puzzle" + to_string(number) + ".txt");
        }
    }

    // Process each requested file with a separate object holding its own
board.
    for (const string &puzzleFile : puzzleFiles) {
        SudokuPuzzle puzzle(puzzleFile);

        // Do not attempt to solve a file that could not be parsed
successfully.
        if (puzzle.loadFromFile()) {
            puzzle.solveAndPrint();
        }
    }

    // A normal return tells the operating system that the program finished.
    return 0;
}
```

Correctness

Input

The programs have odd ways of handling input. For Gemini, it automatically assumes that five files named puzzle1.txt – puzzle5.txt will exist in the same directory that the program is called in. This is incorrect because the files to solve could exist anywhere. The program is not allowed to assume that they will exist in the same directory. ChatGPT has the same issue when not provided input, if the puzzle1.txt - puzzle5.txt exists in the directory. However, it has a very strong method of handling actual arguments. It assumes every argument provided is a path and tries to compute each one. This allows the program to solve multiple problems from a single call and give specific solutions or errors to each one in the order that they are provided. Therefore, on input the ChatGPT program is more correct.

Program Output

The format and verity of both programs is completely correct. The requirements are for each file to output the name of the file, the original puzzle, and all possible solutions. Both programs follow this format, and the solutions that they produce are the correct solutions to the puzzles provided. Therefore, both programs are correct in their output.

Depth-First Recursive Algorithm

Gemini has a perfect implementation of the algorithm described in the instructions, but ChatGPT strays from the method given. Instead of just finding the next empty cell, it searches for the cell with the least candidates and then starts from there. This breaks the rules as there is zero mention of finding the cell with the least candidates in the instructions. This makes Gemini more correct on having a valid depth-first recursive algorithm as opposed to ChatGPT being incorrect.

Execution Time

Algorithm

In general, execution time of both programs is comparable and optimized. The biggest change in execution time for a program like this is how it finds the solution, and that should be decided. Now, as stated in correctness, these programs do not have the same algorithm, but they should. Ignoring correctness, ChatGPT's output should be slightly faster as its method of finding the cell with the least candidates first before evaluating the next empty means that there will be less branches early on which can decrease how many evaluations need to occur. This is because if the first cell checked has 9 candidates, then it is possible that each empty cell afterwards would

need to compute their candidates and solutions 9 times. This would be faster if there were only two candidates for the root branch. Therefore, by this logic, the incorrect ChatGPT program is faster than the correct Gemini program.

Space Complexity

Cell Storage

Both programs have their sudoku classes keep a private 2D 9x9 array of integers as their internal storage. However, ChatGPT's output requires that the board also keep a second 9x9 array as the working board for when the program is trying to create solutions. This is unnecessary and means that each sudoku board requires two 9x9 arrays instead of one, which is a large difference. This makes the way that Gemini stores sudoku cells much less space complex than ChatGPT.

Candidate Listing

ChatGPT has the program create a vector filled with the candidates of each cell and then tries the candidate in a foreach structure. This is opposed to Gemini which simply creates a loop from n = 1 to n = 9 and then tries each option from there. This does not change much in execution time since no matter what the numbers need to be evaluated. However, it does affect space complexity since ChatGPT requires an additional vector to be stored while it computes its solutions. This makes Gemini less space complex in how it evaluates candidates in comparison to ChatGPT.

Maintainability

Comments

Both programs have comments, but they are extremely lackluster. Gemini prefaced all of its comments with "authored by student" even though, they obviously were not. While both programs use comments, they are lackluster and often lacking in depth. For example, it comments the start of a function but doesn't comment any of the other lines within the function. This lack of strong commenting means that the code is harder to read and thus harder to maintain for both programs.

Scope of Sudoku Class

In both programs, the Sudoku class is capable of loading sudoku boards from files. This is problematic because it feels outside of what a sudoku board should care about. Having the sudoku board store the filename and load in files is outside of the scope of what a sudoku board

is about. Were there a separate class that created a sudoku board from a file, then the code would be much more readable and modifiable because there would be a better separation between responsibilities of the program. This increase in readability and modifiability would make the code far more maintainable.

Function Separation

In both programs, function could use more separation of responsibility. The biggest example of this is in both Gemini's "isValid" and ChatGPT's "canPlace" functions that check if a particular cell can take a digit. These functions check if the row is valid, if the column is valid, and if the block is valid. This is an issue because if you wanted to change how the row checking worked, you would need to modify a part of this function. It would be much easier to modify and read the program if the isValid and canPlace functions had separate functions for each check. This would make it much easier to follow each step and would make it easier to know where to change behavior. This issue reduces the maintainability of both programs.

Final Program Choice

In the end, the programs are quite comparable. The biggest factors in my decision were that ChatGPT had better file loading and Gemini had a correct method of solving Sudoku puzzles. For correctness, the better choice is Gemini as correctly solving the Sudoku puzzle according to the requirements is better than incorrect file loading. Therefore, I decided that Gemini's program would be the template that I would base my final program from.

Changes

Better Input

The input for the Gemini input is incorrect as it tries to read 5 files only and only files named puzzle1.txt - puzzle5.txt. Therefore, one of the major changes was using the argument reading concept from ChatGPT. All arguments following the executable will be taken as file paths. By changing this, the correctness of the program increases.

Class Redesigning

The SudokuSolver class was cluttered with file loading and sudoku logic that should have been separated. I created two different classes, SudokuInputReader and SudokuBoard, that properly separated the overburdened class into two neatly sized classes. This increases maintainability as it ensures that it is obvious what class does what and makes trying to modify one aspect of the program easier than if it was a singular god class.

Decomposition

I decomposed a lot of functions that did too many operations into subfunctions that would reduce the complexity in understanding their parent function. This increases maintainability as the functions and their purpose are easier to understand when they are more bite sized and focused.

Comments

I reworked every comment in the program to reflect their author and to explain what every line (or few lines) of code was doing. This makes the program far easier to understand and read, which makes it far more maintainable. Someone who looks at this program after will be able to follow the logic much easier with abundant comments.